T.P. Mini Social Media

11 Novembre, 2026

Bilal Houari
houari.bilal@etu.uae.ac.ma
Master I.D.I.A.

Le but de ce TP est de publier un petit réseau social (`MiniSocial`) sur le réseau de test Ethereum Sepolia, puis d'écrire des messages et de les lire depuis une page web. L'application est surtout un petit contrat Solidity, qui contient la logique de base : écrire un message et lire les messages déjà publiés.

Le contrat

Le contrat `MiniSocial` est très petit. Un message (`Post`) contient le texte et l'adresse de la personne qui l'a écrit, et tous les messages sont rangés dans un tableau. La fonction `publishPost` ajoute un message et prévient la page avec un événement, et `getPost` lit un message avec son numéro. Il y a aussi `getTotalPosts` , qui donne le nombre de messages.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

contract MiniSocial {
    struct Post {
        string message;
        address author;
    }

    Post[] public posts;

    event PostPublished(uint256 indexed index, address indexed author, string message);

    function publishPost(string memory _message) public {
        posts.push(Post(_message, msg.sender));
        emit PostPublished(posts.length - 1, msg.sender, _message);
    }

    function getPost(uint256 index) public view returns (string memory, address) {
        require(index < posts.length, "Post does not exist");
        Post storage post = posts[index];
        return (post.message, post.author);
    }

    function getTotalPosts() public view returns (uint256) {
        return posts.length;
    }
}
```

L'environnement Docker

On nous a recommandé d'utiliser l'éditeur en ligne Remix pour faire le projet, mais j'ai décidé d'utiliser mon éditeur local et de préparer mon environnement dans mon conteneur Docker Kali, que j'avais déjà. Ce conteneur n'est pas dans le dépôt Git, parce que ce n'est pas le but du TP, mais il contient déjà les outils dont j'ai besoin : j'ai installé Foundry, etc.

Comme Foundry marche dans le terminal, je peux compiler et tester avec `forge build` et `forge test`, puis déployer avec `forge script`. Les tests et le script de déploiement restent avec le contrat, et je peux refaire le déploiement facilement. Pour la page web, c'est un autre projet : c'est du React, et Vite lance le serveur. Le `compose.yaml` a aussi un service `anvil`, que je présente plus loin.

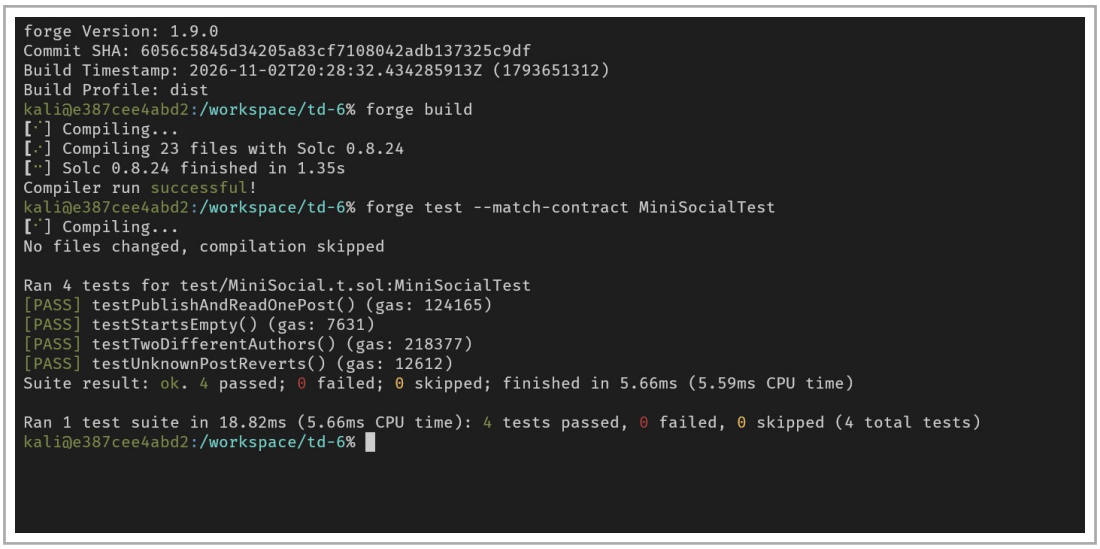


Fig. 1. – La compilation et les tests avec Foundry.

Portefeuilles locaux et Enkrypt

Pour les comptes de test, je n'utilise pas d'extension de portefeuille pour les créer : je génère les clés localement, dans `.wallets/wallets.json`, et j'importe seulement celles dont j'ai besoin dans Enkrypt. Ce fichier n'est pas dans Git, donc les clés ne partent pas dans le Github. Le compte qui déploie le contrat est un compte jetable, que je garde juste au cas où. Enkrypt sert seulement de pont entre ces comptes locaux et le navigateur.

Déploiement sur Sepolia

Je compile et je déploie le contrat avec Foundry. Le compte de déploiement est gardé sous forme chiffrée dans `.deployer/`, un dossier que Git ignore aussi, donc il n'y a pas de clé privée en clair dans le code.

```
forge script script/MiniSocial.s.sol:MiniSocialScript \
  --rpc-url https://ethereum-sepolia-rpc.publicnode.com \
  --keystore .deployer/keystores/sepolia-deployer --broadcast
```

```
kali@e387cee4abd2:/workspace/td-6% forge script script/MiniSocial.s.sol:MiniSocialScript \
  --rpc-url https://ethereum-sepolia-rpc.publicnode.com \
  --keystore .deployer/keystores/sepolia-deployer --broadcast
Enter keystore password:
[⠊] Compiling...
No files changed, compilation skipped
Script ran successfully.

== Return ==
social: contract MiniSocial 0xA057ef59E8927CE59C89895211dbF7E458C77830

== Logs ==
  MiniSocial deployed at: 0xA057ef59E8927CE59C89895211dbF7E458C77830

## Setting up 1 EVM.

==========================

Chain 11155111

Estimated max fee per gas: 0.000193763 gwei
Estimated base fee per gas: 0.000000019 gwei
Estimated max priority fee per gas: 0.000193725 gwei

Estimated total gas used for script: 3549078

Estimated amount required: 0.000000687680000514 ETH

==========================

##### sepolia
✅  [Success] Hash: 0xf0a6ba3172c953bd0d950e556567de8923909f96f0dc05488a1043886a75c70c
Contract: MiniSocial
Contract Address: 0xA057ef59E8927CE59C89895211dbF7E458C77830
Block: 12119855
Paid: 0.00000242277536664 ETH (2730060 gas * 0.000887444 gwei)

✅ Sequence #1 on sepolia | Total Paid: 0.00000242277536664 ETH (2730060 gas * avg 0.000887444 gwei)


==========================

ONCHAIN EXECUTION COMPLETE & SUCCESSFUL.

Transactions saved to: /workspace/td-6/broadcast/MiniSocial.s.sol/11155111/run-latest.json

Sensitive values saved to: /workspace/td-6/cache/MiniSocial.s.sol/11155111/run-latest.json

kali@e387cee4abd2:/workspace/td-6%
```

Fig. 2. – Le déploiement avec `forge script`.

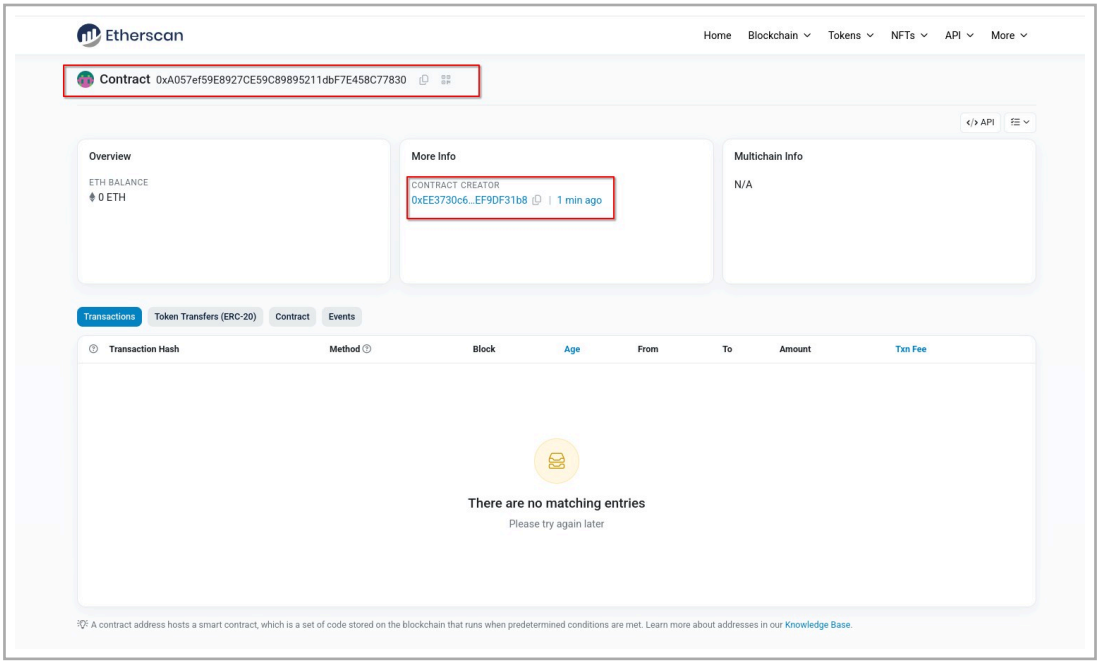


Fig. 3. – Le contrat est sur Sepolia Etherscan.

Publication depuis la page web

Une fois le contrat déployé, la page permet de connecter un compte local avec Enkrypt, d'écrire un message et de lire la liste des messages, qui viennent directement de la blockchain. Quand je publie un message, Enkrypt demande de confirmer la transaction, puis la page relit la liste.

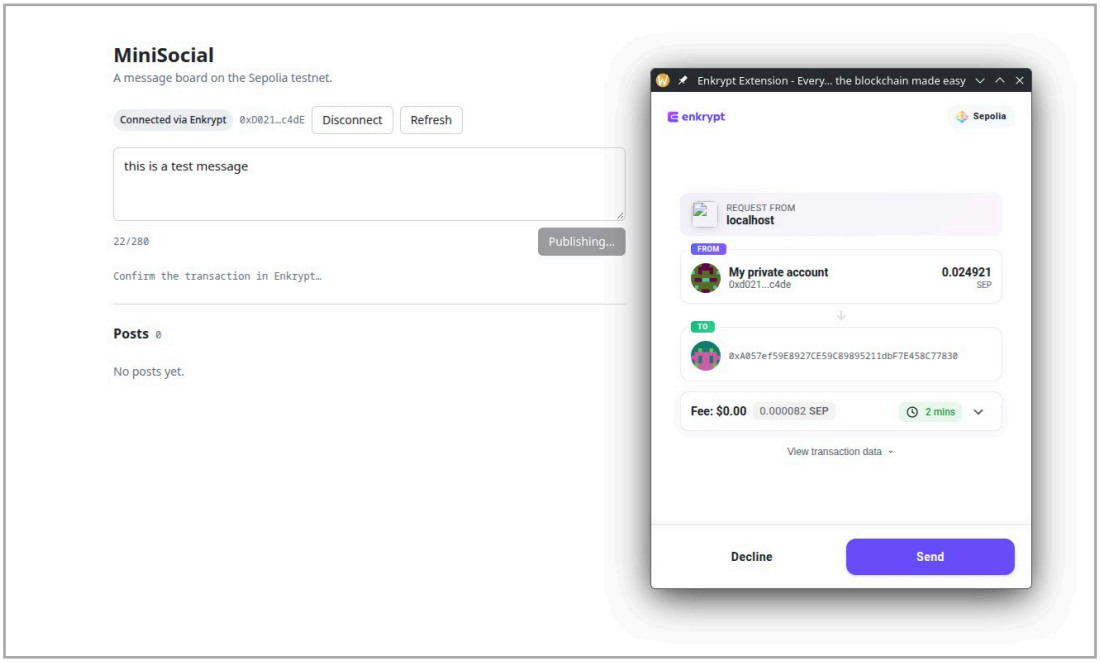


Fig. 4. – Je confirme la transaction dans Enkrypt.

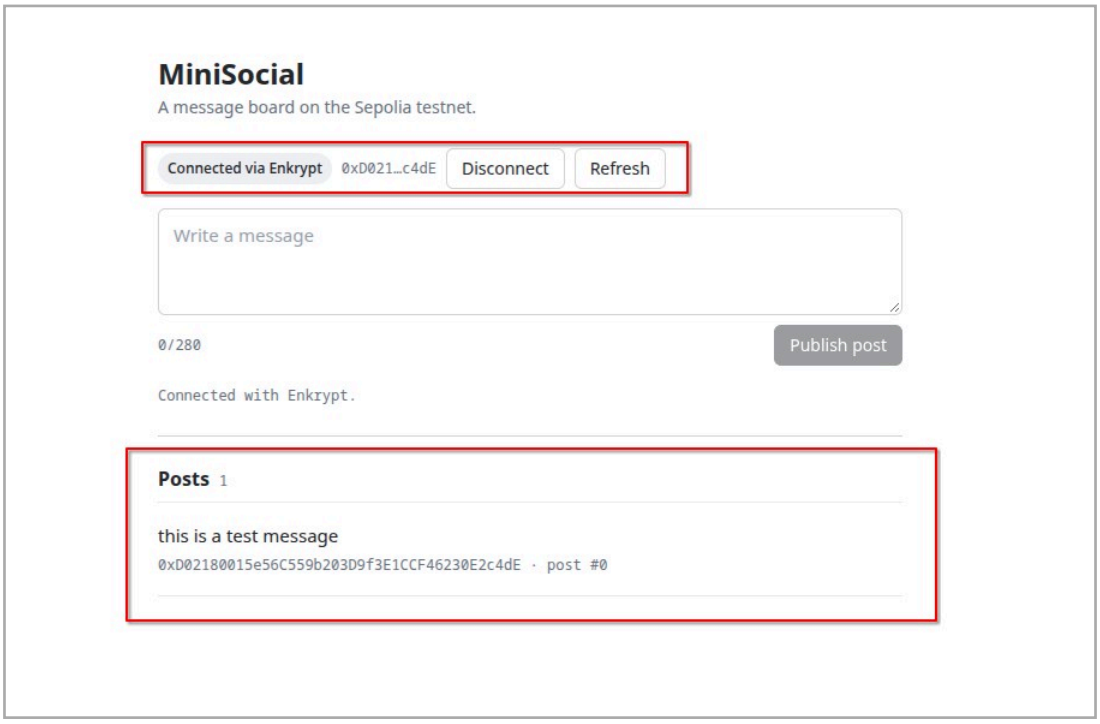


Fig. 5. – Le message est publié, et la page le lit depuis la blockchain.

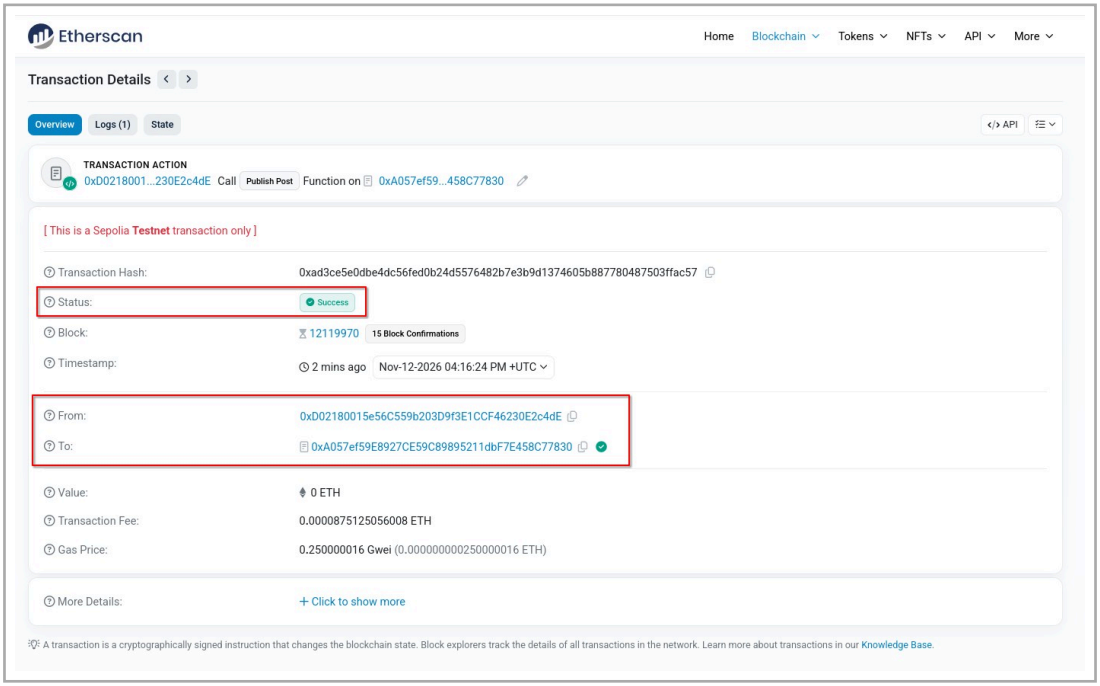


Fig. 6. – La transaction `publishPost` est confirmée sur Etherscan.

Autre solution : Anvil

Sepolia a besoin d'un RPC public et d'un faucet, ce qui n'est pas toujours pratique. Le même `compose.yaml` a donc un service Anvil, qui donne une blockchain locale où les comptes ont déjà de l'argent.

```
docker compose --profile blockchain up -d anvil
```

Anvil écoute sur le port `8545` (chain ID `31337`) : depuis ma machine, l'adresse est `http://127.0.0.1:8545`, et dans le conteneur, c'est `http://anvil:8545`. Pour l'utiliser, il suffit de changer `VITE_RPC_URL`, de déployer le contrat encore une fois et d'ajouter ce réseau dans Enkrypt. La chaîne repart de zéro à chaque redémarrage, donc les messages ne restent pas. Dans ce rapport, les captures sont sur Sepolia, mais avec Anvil je peux travailler sans faucet et sans RPC public.

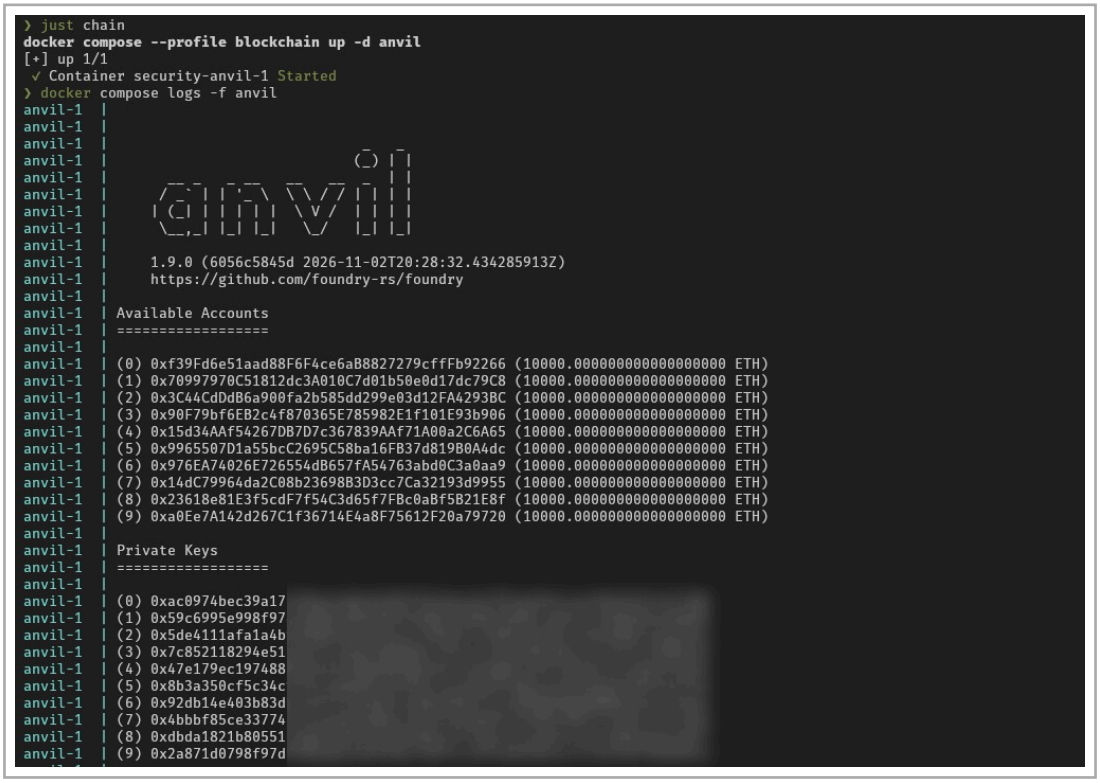


Fig. 7. – Anvil remplace Sepolia sur ma machine.